

UNIVERSITÀ
POLITECNICA
DELLE MARCHE

Corso di «Ingegneria del Software»

A.A. 2024/2025

Parte X: Ingegneria del Software Avanzata

Prof. Domenico Ursino

d.ursino@univpm.it

Component-based Development Paradigm

- La Component Based Software Engineering (CBSE), chiamata anche Component Based Development (CBD), è una branca dell'ingegneria del software che enfatizza la tecnica della suddivisione delle attività rispetto alla vasta gamma di funzionalità disponibili in un dato sistema software.

- È un approccio basato sul riutilizzo per definire, implementare e comporre componenti indipendenti liberamente accoppiati in sistemi.

- Questa pratica mira a portare un grado altrettanto ampio di benefici sia a breve che a lungo termine per il software stesso e per le organizzazioni che sponsorizzano tale software.

- I professionisti dell'ingegneria del software considerano i componenti come parte della piattaforma di partenza per l'orientamento ai servizi.

- I componenti giocano questo ruolo, per esempio, nei web services e, più recentemente nella Service Oriented Architecture (SOA), dove un componente è convertito dal servizio web in un servizio e successivamente eredita ulteriori caratteristiche oltre a quelle di un componente ordinario.

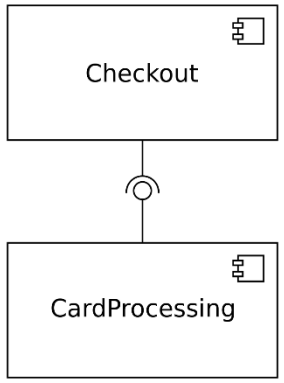
- I componenti possono produrre o consumare eventi e possono essere usati per Event Driven Architecture (EDA).

Component-based Development Paradigm

- Un singolo componente software è un package, un web service, una risorsa web o un modulo che incapsula un insieme di funzioni (o dati) correlati.
- Tutti i processi di sistema sono collocati in componenti separati in modo che tutti i dati e le funzioni all'interno di ogni componente siano semanticamente correlati (proprio come il contenuto delle classi).
- A causa di questo principio, si dice spesso che i componenti sono modulari e coesi.
- Per quanto riguarda il coordinamento a livello di sistema, i componenti comunicano tra loro tramite interfacce.
- Quando un componente offre servizi al resto del sistema, adotta un'interfaccia fornita che specifica i servizi che gli altri componenti possono utilizzare, e come possono farlo.
- Questa interfaccia può essere vista come una firma del componente - il cliente non ha bisogno di conoscere il funzionamento interno del componente (implementazione) per poterlo utilizzare.
- Questo principio va sotto il nome di incapsulamento nei componenti.
- Quando un componente ha bisogno di usare un altro componente per funzionare, adotta un'interfaccia usata che specifica i servizi di cui ha bisogno.

Component-based Development Paradigm

- Un esempio di questo modo di procedere viene riportato nella seguente figura:



- Il componente *Checkout*, responsabile della gestione dell'ordine di acquisto di un cliente, richiede al componente *CardProcessing* di effettuare l'opportuno addebito sulla carta di credito/debito del cliente.

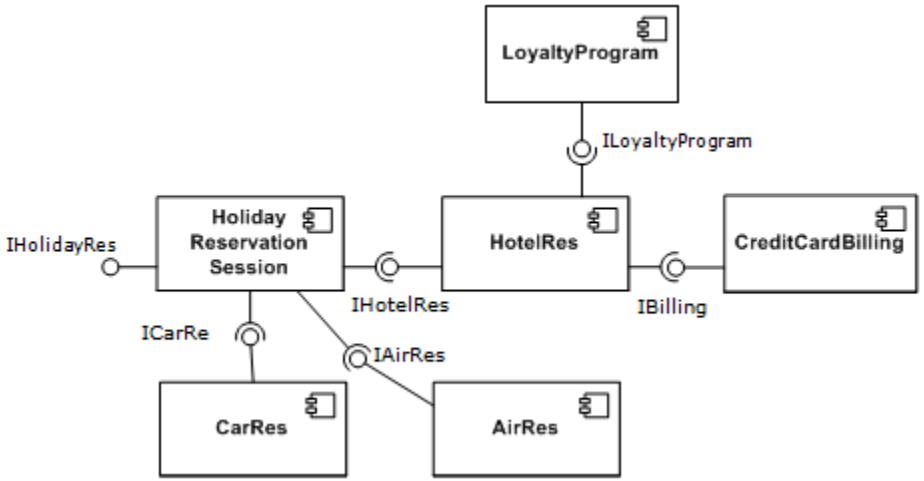
- Un altro importante attributo dei componenti è che sono sostituibili, in modo tale che un componente può sostituirne un altro (in fase di progettazione o di esecuzione), se il nuovo componente soddisfa i requisiti del vecchio (espressi attraverso le interfacce).

- Di conseguenza, i componenti possono essere sostituiti con una versione aggiornata o un'alternativa senza rompere il sistema in cui essi operano.

Component-based Development Paradigm

- Come regola empirica per gli ingegneri che sostituiscono i componenti, il componente B può immediatamente sostituire il componente A, se il componente B fornisce almeno ciò che il componente A forniva e utilizza non più di ciò che il componente A utilizzava.

- Un semplice esempio di diversi componenti software che rappresentano un ipotetico sistema di prenotazione delle vacanze viene riportato nella seguente figura:



- La riusabilità è una caratteristica importante di un componente software di alta qualità.

- I programmatori dovrebbero progettare e implementare i componenti software in modo tale che molti programmi diversi possano riutilizzarli.

Component-based Development Paradigm

- Inoltre, è necessario effettuare test di usabilità basati sui componenti quando i componenti software interagiscono direttamente con gli utenti.

- Ci vogliono sforzi significativi e consapevolezza per scrivere un componente software che sia effettivamente riutilizzabile.

- Il componente deve essere:

  - completamente documentato;

  - accuratamente testato;

  - robusto - con un controllo completo della validità dell'input capace di restituire appropriati messaggi di errore o codici di ritorno progettato con la consapevolezza che sarà utilizzato per usi imprevisti.

- A partire dal 2010, i moderni componenti riutilizzabili incapsulano sia le strutture di dati che gli algoritmi che sono applicati alle strutture di dati.

- L'ingegneria del software basata sui componenti si basa sulle teorie precedenti degli oggetti software, delle architetture software, dei framework software e dei design pattern, e sull'estesa teoria della programmazione orientata agli oggetti.

Component-based Development Paradigm

- Essa sostiene che i componenti software, come i componenti hardware, usati per esempio nelle telecomunicazioni, possono alla fine essere resi intercambiabili e affidabili.
- D'altra parte, si sostiene che è un errore concentrarsi sui componenti indipendenti piuttosto che sul framework (senza il quale non esisterebbero).
- Un computer che esegue diversi componenti software è spesso chiamato application server.
- Questa combinazione di application server e componenti software è solitamente chiamata calcolo distribuito.
- Una tipica applicazione del mondo reale di questo è, ad esempio, nelle applicazioni finanziarie o nel software aziendale.
- Un component model è una definizione di proprietà che i componenti devono soddisfare, nonché di metodi e meccanismi per la composizione dei componenti.
- Negli ultimi decenni, ricercatori e professionisti hanno proposto diversi component model con caratteristiche diverse.
- Esempi di component model sono Il modello Enterprise JavaBeans (EJB) e il modello .NET.

Distributed computing

- Il distributed computing è un campo dell'informatica che studia i sistemi distribuiti.
- Un sistema distribuito è un sistema i cui componenti si trovano su diversi computer in rete, che comunicano e coordinano le loro azioni passando messaggi l'uno all'altro da qualsiasi sistema.
- I componenti interagiscono tra loro per raggiungere un obiettivo comune.
- Tre caratteristiche significative dei sistemi distribuiti sono:
  - la concorrenza dei componenti;
  - la mancanza di un clock globale;
  - il fallimento indipendente dei componenti, ovvero la possibilità che quando i componenti di un sistema falliscono, essi non portano al fallimento dell'intero sistema.
- Esempi di sistemi distribuiti variano dai sistemi basati su SOA ai giochi online massivamente multiplayer alle applicazioni peer-to-peer.
- Un programma per computer che gira all'interno di un sistema distribuito è chiamato programma distribuito (e la programmazione distribuita è il processo di scrivere tali programmi).

Distributed computing

- Ci sono molti tipi diversi di implementazioni per il meccanismo di passaggio dei messaggi, tra cui HTTP puro, connettori simili a RPC e code di messaggi.

- Il distributed computing si riferisce anche all'uso di sistemi distribuiti per risolvere problemi computazionali.

- In questo caso, un problema è diviso in molti task, ognuno dei quali è risolto da uno o più computer, che comunicano tra loro attraverso il passaggio di messaggi.

Distributed computing - Introduzione

- La parola «distribuito» in termini come "sistema distribuito", "programmazione distribuita" e "algoritmo distribuito" originariamente si riferiva a reti di computer in cui i singoli computer erano fisicamente distribuiti all'interno di una certa area geografica.
- I termini sono oggi utilizzati in un senso molto più ampio, anche riferendosi a processi autonomi che girano sullo stesso computer fisico e interagiscono tra loro tramite il passaggio di messaggi.
- Mentre non esiste una definizione unica di sistema distribuito, le seguenti proprietà caratterizzano un sistema distribuito:
  - Ci sono diverse entità computazionali autonome (computer o nodi), ognuna delle quali ha la propria memoria locale.
  - Le entità comunicano tra loro tramite il passaggio di messaggi.
- Un sistema distribuito può avere un obiettivo comune, come la risoluzione di un grande problema computazionale.
- L'utente percepisce quindi l'insieme dei processori autonomi come un'unità.
- In alternativa, ogni computer può avere un proprio utente con esigenze individuali, e lo scopo del sistema distribuito è quello di coordinare l'uso delle risorse condivise o fornire servizi di comunicazione agli utenti.

Distributed computing - Introduzione

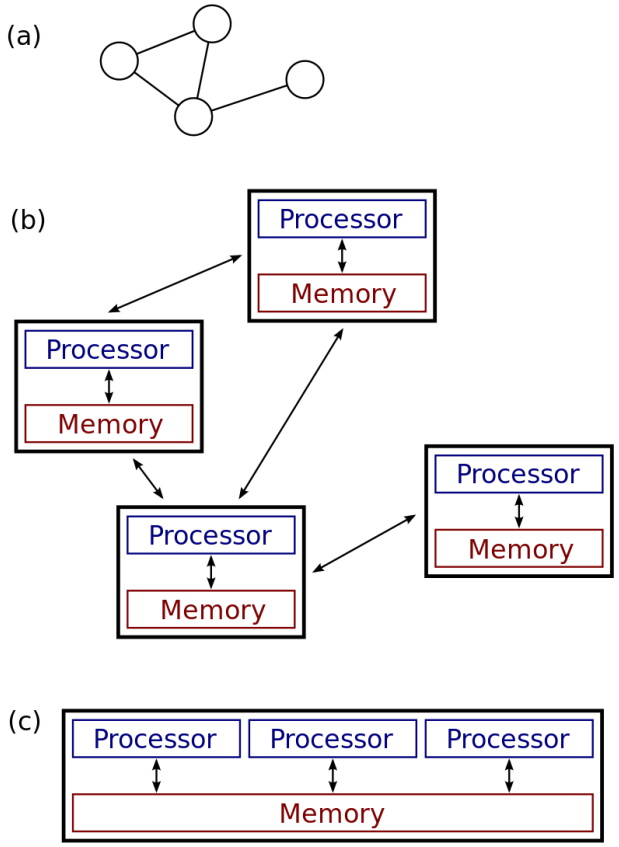
- Altre proprietà tipiche dei sistemi distribuiti sono le seguenti:
  - Il sistema deve tollerare guasti nei singoli computer.
  - La struttura del sistema (topologia di rete, latenza di rete, numero di computer) non è nota in anticipo, il sistema può essere composto da diversi tipi di computer e collegamenti di rete, e il sistema può cambiare durante l'esecuzione di un programma distribuito.
  - Ogni computer ha solo una visione limitata e incompleta del sistema; in altra parole, ogni computer può conoscere solo una parte dell'input.

Distributed computing – Calcolo parallelo e distribuito

- I sistemi distribuiti sono gruppi di computer in rete che condividono un obiettivo comune per il loro lavoro.
- I termini "calcolo concorrente", "calcolo parallelo" e "calcolo distribuito" si sovrappongono molto e non esiste una chiara distinzione tra loro.
- Lo stesso sistema può essere caratterizzato sia come "parallelo" che come "distribuito"; i processori in un tipico sistema distribuito girano simultaneamente in parallelo.
- Il calcolo parallelo può essere visto come una particolare forma strettamente accoppiata di calcolo distribuito, e il calcolo distribuito può essere visto come una forma liberamente accoppiata di calcolo parallelo.
- Tuttavia, è possibile classificare approssimativamente i sistemi concorrenti come "paralleli" o "distribuiti" utilizzando i seguenti criteri:
  - Nel calcolo parallelo, tutti i processori possono avere accesso a una memoria condivisa per scambiare informazioni tra di essi.
  - Nel calcolo distribuito, ogni processore ha la sua memoria privata (memoria distribuita) e le informazioni vengono scambiate attraverso lo scambio di messaggi.

Distributed computing – Calcolo parallelo e distribuito

- La seguente figura illustra la differenza tra sistemi distribuiti e paralleli.



- La figura (a) è una vista schematica di un tipico sistema distribuito; il sistema è rappresentato come una topologia di rete in cui ogni nodo è un computer e ogni linea che collega i nodi è un collegamento di comunicazione.

Distributed computing – Calcolo parallelo e distribuito

- La figura (b) mostra lo stesso sistema distribuito in modo più dettagliato: ogni computer ha la sua memoria locale, e le informazioni possono essere scambiate solo passando messaggi da un nodo all'altro usando i collegamenti di comunicazione disponibili.

- La figura (c) mostra un sistema parallelo in cui ogni processore ha un accesso diretto a una memoria condivisa.

- Come regola generale, il calcolo parallelo ad alte prestazioni in un multiprocessore a memoria condivisa usa algoritmi paralleli mentre il coordinamento di un sistema distribuito su larga scala usa algoritmi distribuiti.

Distributed computing – Architetture

- Varie architetture hardware e software sono usate per il calcolo distribuito.
- Ad un livello più basso, è necessario interconnettere più CPU con un qualche tipo di rete, indipendentemente dal fatto che questa rete sia stampata su un circuito o fatta di dispositivi e cavi liberamente accoppiati.
- Ad un livello più alto, è necessario interconnettere i processi in esecuzione su quelle CPU con qualche tipo di sistema di comunicazione.
- La programmazione distribuita rientra tipicamente in una delle seguenti diverse architetture di base:
  - Client-server: architetture in cui client intelligenti contattano il server per i dati, poi li formattano e li mostrano agli utenti.
  - Three-tier: architetture che spostano l'intelligenza del client su un livello intermedio in modo che possano essere usati client senza stato.
    - Questo semplifica la distribuzione delle applicazioni.
    - La maggior parte delle applicazioni web sono a three-tier.

Distributed computing – Architetture

- n-tier: architetture che si riferiscono tipicamente ad applicazioni web che inoltrano ulteriormente le loro richieste ad altri servizi aziendali.
    - Questo tipo di applicazione è quello più responsabile del successo degli application server.
- Peer-to-peer: architetture in cui non ci sono macchine speciali che forniscono un servizio o gestiscono le risorse di rete.
    - Invece tutte le responsabilità sono divise uniformemente tra tutte le macchine, note come peer.
    - I peer possono servire sia come client che come server.
    - Esempi di questa architettura includono BitTorrent e la rete Bitcoin.
- Un altro aspetto fondamentale dell'architettura di calcolo distribuito è il metodo di comunicazione e coordinamento del lavoro tra processi concorrenti.
- Attraverso vari protocolli di passaggio di messaggi, i processi possono comunicare direttamente tra loro, tipicamente in una relazione master/slave.

Distributed computing – Architetture

- In alternativa, un'architettura "database-centrica" può permettere che il calcolo distribuito sia fatto senza alcuna forma di comunicazione diretta tra i processi, utilizzando un database condiviso.

Distributed computing – Applicazioni

- Le ragioni per usare sistemi distribuiti e il distributed computing sono come di seguito specificato:
  - La natura stessa di un'applicazione può richiedere l'uso di una rete di comunicazione che colleghi diversi computer: per esempio, dati prodotti in un luogo fisico e richiesti in un altro luogo.
  - Ci sono molti casi in cui l'uso di un singolo computer sarebbe possibile in linea di principio, ma l'uso di un sistema distribuito è vantaggioso per ragioni pratiche.
    - Per esempio, può essere più efficiente in termini di costi ottenere il livello di prestazioni desiderato utilizzando un cluster di diversi computer di fascia bassa, rispetto a un singolo computer di fascia alta.
    - Un sistema distribuito può fornire più affidabilità di un sistema non distribuito, poiché non c'è un singolo punto di failure.
    - Inoltre, un sistema distribuito può essere più facile da espandere e gestire di un sistema monolitico uniprocessore.

Distributed computing – Esempi

- Esempi di sistemi distribuiti e applicazioni di calcolo distribuito includono i seguenti:
  - reti di telecomunicazioni:
    - reti telefoniche e reti cellulari,
    - reti di computer come Internet,
    - reti di sensori wireless,
    - algoritmi di routing;
  - applicazioni di rete:
    - World Wide Web e reti peer-to-peer,
    - giochi online multiplayer di massa e comunità di realtà virtuale,
    - database distribuiti e sistemi di gestione di database distribuiti,

Distributed computing – Esempi

  - file system di rete,
  - cache distribuite come i burst buffer,
  - sistemi distribuiti di elaborazione delle informazioni come i sistemi bancari e i sistemi di prenotazione aerea;
- controllo dei processi in tempo reale:
  - sistemi di controllo per aerei,
  - sistemi di controllo industriale;
- calcolo parallelo:
  - calcolo scientifico, tra cui cluster computing, grid computing, cloud computing, e vari progetti di calcolo volontario,
  - rendering distribuito in computer graphics.

Service Oriented Architecture - Introduzione

- La Service Oriented Architecture (SOA) è uno stile architetturale che supporta l'orientamento ai servizi.
- Di conseguenza, è applicata in quei casi in cui dei componenti di un'applicazione forniscono dei servizi ad altri componenti attraverso un protocollo di comunicazione su una rete.
- Un servizio è un'unità discreta di funzionalità a cui si può accedere da remoto, su cui si può agire e che può essere aggiornata in modo indipendente come, ad esempio, recuperare un estratto conto della carta di credito online.
- La SOA è anche pensata per essere indipendente da fornitori, prodotti e tecnologie.
- L'orientamento ai servizi è un modo di pensare e organizzare le cose basandosi sui servizi.
- Un servizio ha quattro proprietà:
  - Rappresenta logicamente un'attività di business ripetibile con un risultato specifico.
  - È autocontenuto.
  - È una scatola nera per i suoi consumatori, il che significa che il consumatore non deve essere consapevole del funzionamento interno del servizio.

Service Oriented Architecture - Introduzione

  - Può essere composto da altri servizi.

- Diversi servizi possono essere usati insieme come una rete di servizi per fornire la funzionalità di una grande applicazione software, un principio che la SOA condivide con la programmazione modulare.

- L'architettura orientata ai servizi integra componenti software distribuiti, mantenuti e distribuiti separatamente.

- Essa è supportata da tecnologie e standard che facilitano la comunicazione e la cooperazione dei componenti su una rete, specialmente su una rete IP.

- La SOA è legata all'idea di un'Application Programming Interface (API), un'interfaccia o un protocollo di comunicazione tra diverse parti di un programma per computer destinato a semplificare l'implementazione e la manutenzione del software.

- Un'API può essere pensata come il servizio, e la SOA può essere vista come l'architettura che permette al servizio di funzionare.

- Nella SOA i servizi usano protocolli che descrivono come passano e analizzano i messaggi usando metadati di descrizione.

Service Oriented Architecture - Introduzione

- Questi metadati descrivono sia le caratteristiche funzionali che le caratteristiche di qualità del servizio.
- L'architettura orientata ai servizi mira a permettere agli utenti di combinare funzionalità per formare applicazioni che sono costruite puramente da servizi esistenti e combinandoli in un modo ad hoc.
- Un servizio presenta una semplice interfaccia al richiedente che astrae la complessità sottostante agendo come una scatola nera.
- Ulteriori utenti possono anche accedere a questi servizi indipendenti senza alcuna conoscenza della loro implementazione interna.

Service Oriented Architecture – Concetti di base

- La parola d'ordine che la service-orientation promuove è l'accoppiamento libero tra i servizi.
- La SOA separa le funzioni in unità distinte, o servizi, che gli sviluppatori rendono accessibili su una rete per permettere agli utenti di combinarli e riutilizzarli nella produzione di applicazioni.
- Questi servizi e i loro corrispondenti consumatori comunicano tra loro passando dati in un formato ben definito e condiviso, o coordinando un'attività tra due o più servizi.
- Nell'ottobre 2009 è stato pubblicato un manifesto per la Service Oriented Architecture.
- Questo prevede sei valori fondamentali che sono elencati come segue:
  - Si dà più importanza al valore del business che alla strategia tecnica.
  - Si dà più importanza agli obiettivi strategici che ai benefici specifici del progetto.
  - Si dà più importanza all'interoperabilità intrinseca che all'integrazione personalizzata.
  - Si dà più importanza ai servizi condivisi che alle implementazioni per scopi specifici.

Service Oriented Architecture – Concetti di base

  - Si dà più importanza alla flessibilità che all'ottimizzazione.
  - Si dà più importanza al perfezionamento evolutivo che alla ricerca della perfezione iniziale.

- La SOA può essere vista come parte del continuum che va dal vecchio concetto di calcolo distribuito e di programmazione modulare, attraverso la SOA, fino alle pratiche di mashup, SaaS e cloud computing (che alcuni vedono come figli della SOA).

Service Oriented Architecture – Principi

- Non ci sono standard industriali relativi all'esatta composizione di un'architettura orientata ai servizi, sebbene molte fonti industriali abbiano pubblicato i loro principi.

- Alcuni di questi principi sono i seguenti:

  - Contratto di servizio standardizzato:

    - I servizi aderiscono ad un accordo di comunicazione standard, come definito collettivamente da uno o più documenti di descrizione del servizio all'interno di un dato insieme di servizi.

  - Autonomia di riferimento del servizio (un aspetto dell'accoppiamento lasco):

    - La relazione tra i servizi è minimizzata al livello in cui essi sono solo consapevoli della loro esistenza.

  - Trasparenza della posizione del servizio (un aspetto dell'accoppiamento lasco):

    - I servizi possono essere chiamati da qualsiasi punto della rete in cui ci si trova, non importa dove essi siano realmente presenti.

Service Oriented Architecture – Principi

- Longevità del servizio:
  - I servizi dovrebbero essere progettati per durare a lungo.
  - Dove possibile i servizi dovrebbero evitare di forzare i consumatori a cambiare se non richiedono nuove caratteristiche.
  - Se si chiama un servizio oggi si dovrebbe essere in grado di chiamare lo stesso servizio domani.
- Astrazione del servizio:
  - I servizi agiscono come scatole nere, cioè la loro logica interna è nascosta ai consumatori.
- Autonomia del servizio:
  - I servizi sono indipendenti e controllano le funzionalità che incapsulano, sia dal punto di vista del Design-time che del run-time.

Service Oriented Architecture – Principi

- Apolidia del servizio:
  - I servizi sono apolidi, cioè restituiscono il valore richiesto o danno un'eccezione, minimizzando così l'uso delle risorse.
- Granularità del servizio:
  - Un principio per assicurare che i servizi abbiano una dimensione e un ambito adeguati.
  - La funzionalità fornita dal servizio all'utente deve essere rilevante.
- Normalizzazione dei servizi:
  - I servizi sono decomposti o consolidati (normalizzati) per minimizzare la ridondanza.
  - In alcuni casi, tale minimizzazione non può avvenire. Questi sono i casi in cui sono richiesti l'ottimizzazione delle prestazioni, l'accesso e l'aggregazione.

Service Oriented Architecture – Principi

- Possibilità di comporre i servizi.
  - I servizi possono essere usati per comporre altri servizi.
- Scoperta di servizi.
  - I servizi sono integrati da metadati grazie ai quali possono essere efficacemente scoperti e interpretati.
- Riutilizzabilità dei servizi.
  - La logica è divisa in vari servizi, per promuovere il riutilizzo del codice.
- Incapsulamento dei servizi
  - Molti servizi che non erano inizialmente previsti per la SOA, possono essere incapsulati o diventare parte della SOA.

Service Oriented Architecture – Funzionamento

- Ogni building block della SOA può giocare uno qualsiasi dei tre ruoli:
  - Fornitore di servizi:
    - Crea un servizio web e fornisce le sue informazioni al registro dei servizi.
    - Ogni fornitore deve gestire problematiche quali il come e il perché: quale servizio esporre, a quale dare più importanza, se privilegiare la sicurezza o la disponibilità, a quale prezzo offrire il servizio e molto altro ancora.
    - Il fornitore deve anche decidere in quale categoria il servizio dovrebbe essere elencato per un dato servizio di broker e che tipo di accordi con i partner commerciali sono richiesti per usare il servizio.
  - Broker di servizi, registro di servizi o deposito di servizi:
    - La sua funzionalità principale è quella di rendere le informazioni riguardanti il servizio web disponibili a qualsiasi potenziale richiedente.
    - Chi implementa il broker decide lo scopo del broker.

Service Oriented Architecture – Funzionamento

- I broker pubblici sono disponibili ovunque e dappertutto, ma i broker privati sono disponibili solo per una quantità limitata di pubblico.
- UDDI è stato un primo tentativo, non più attivamente supportato, di supportare la scoperta dei servizi web.

- Richiedente/consumatore di servizi:
  - Individua le voci nel registro dei broker usando varie operazioni di ricerca e poi si collega al fornitore di servizi per invocare uno dei suoi servizi web.
  - Qualsiasi sia il servizio di cui hanno bisogno i consumatori, questi devono individuarlo nel broker, collegarlo al proprio servizio e poi usarlo.
  - Essi possono accedere a più servizi se ne hanno bisogno e il broker li fornisce.

- La relazione consumatore-provider del servizio è governata da un contratto di servizio standardizzato, che ha una parte commerciale, una parte funzionale e una parte tecnica.

Service Oriented Architecture – Funzionamento

- I pattern di composizione dei servizi hanno due principali stili architetturali di alto livello: coreografia e orchestrazione.

- Pattern di integrazione di livello inferiore che non sono legati ad un particolare stile architetturale sono comunque ammissibili e possono comunque essere utilizzati nella progettazione SOA.

Real-time Computing – Introduzione

- Real-time computing (RTC) è il termine informatico per indicare i sistemi hardware e software soggetti a un "vincolo in tempo reale", per esempio dall'evento alla risposta del sistema.
- I programmi in tempo reale devono garantire la risposta entro determinati vincoli di tempo, spesso indicati come «deadline».
- Le risposte in tempo reale spesso devono essere fornite in un tempo dell'ordine dei millisecondi, e talvolta dei microsecondi.
- Un sistema non specificato come operante in tempo reale di solito non può garantire una risposta in qualsiasi lasso di tempo, anche se possono essere forniti tempi di risposta *tipici* o *previsti*.
- L'elaborazione in tempo reale *fallisce* se non viene completata entro una scadenza specificata rispetto a un evento; le scadenze devono essere sempre rispettate, indipendentemente dal carico del sistema.
- Un sistema in tempo reale è stato descritto come un sistema che "controlla un ambiente ricevendo dati, elaborandoli e restituendo i risultati in modo sufficientemente rapido da influenzare l'ambiente in quel momento".
- Il termine "tempo reale" è anche usato nella simulazione per significare che il clock della simulazione funziona alla stessa velocità di un clock reale, nonché nel controllo dei processi e nei sistemi aziendali per significare "senza ritardi significativi".

Real-time Computing – Introduzione

- Il software in tempo reale può utilizzare uno o più dei seguenti strumenti, che forniscono le strutture essenziali su cui costruire un'applicazione software in tempo reale:
  - linguaggi di programmazione sincroni;
  - sistemi operativi in tempo reale;
  - reti in tempo reale.
- I sistemi usati per molte applicazioni critiche per la sicurezza devono essere in tempo reale, come le applicazioni per il controllo degli aerei o dei freni antibloccaggio, entrambi i quali richiedono una risposta meccanica immediata e precisa.

Real-time Computing – Criteri

- Un sistema è detto in tempo reale se la correttezza totale di un'operazione dipende non solo dalla sua correttezza logica, ma anche dal tempo in cui viene eseguita.

- I sistemi in tempo reale, così come le loro deadline, sono classificati in base alle conseguenze che vi sono se non viene rispettata una deadline. In particolare, abbiamo i seguenti sistemi:

  - Hard: in questo caso il mancato rispetto di una deadline porta al fallimento totale del sistema.

  - Firm: in questo caso i mancati rispetti di una deadline sono poco frequenti e tollerabili, ma possono degradare la qualità del servizio del sistema; infatti, l'utilità di un risultato è zero se arriva dopo la deadline.

  - Soft: in questo caso l'utilità di un risultato degrada dopo la deadline, degradando così la qualità del servizio del sistema.

- Così, l'obiettivo di un sistema hard real-time è quello di garantire che tutte le deadline siano rispettate, mentre per i sistemi soft real-time l'obiettivo diventa quello di rispettare un certo sottoinsieme di deadline al fine di ottimizzare alcuni criteri specifici dell'applicazione.

- I criteri particolari ottimizzati dipendono dall'applicazione, ma alcuni esempi tipici includono la massimizzazione del numero di deadline rispettate, la minimizzazione del ritardo dei task e la massimizzazione del numero di task ad alta priorità che rispettano le loro deadline.

Real-time Computing – Criteri

- I sistemi hard real-time sono utilizzati quando è indispensabile che ad un evento si reagisca entro una deadline rigorosa.

- Queste forti garanzie sono richieste da sistemi per i quali non reagire in un certo intervallo di tempo causerebbe grandi perdite in qualche modo, specialmente danneggiando fisicamente l'ambiente circostante o minacciando vite umane (anche se la definizione stretta è semplicemente che mancare la deadline costituisce un fallimento del sistema).

- Alcuni esempi di sistemi hard real-time sono i seguenti:

  - Un sistema di controllo del motore di un'automobile è un sistema hard real-time perché un segnale ritardato può causare un guasto o un danno al motore.

  - Sistemi medici come i pacemaker sono hard real-time.

    - Anche se il task di un pacemaker è semplice, a causa del rischio potenziale per la vita umana, i sistemi medici come questi sono tipicamente soggetti a test approfonditi e alla certificazione, che a sua volta richiede un hard real-time computing al fine di offrire garanzie dimostrabili che un fallimento è improbabile o impossibile.

Real-time Computing – Criteri

- I controllori di processi industriali, come una macchina su una linea di assemblaggio, sono sistemi hard real-time.
  - Se la macchina è in ritardo, l'articolo sulla linea di assemblaggio potrebbe passare oltre la portata della macchina (lasciando il prodotto non elaborato), oppure la macchina o il prodotto potrebbero essere danneggiati attivando il robot nel momento sbagliato.
  - Se il guasto viene rilevato, entrambi i casi porterebbero all'arresto della catena di montaggio, il che rallenta la produzione.
  - Se il guasto non viene rilevato, un prodotto con un difetto potrebbe passare attraverso la produzione, o potrebbe causare danni in fasi successive della produzione.
- Sistemi hard real-time si trovano tipicamente ad interagire ad un basso livello con l'hardware fisico, nei sistemi embedded.
  - I primi sistemi di videogiochi come l'Atari 2600 e la grafica vettoriale Cinematronics avevano requisiti hard real-time a causa della natura della grafica e dell'hardware di temporizzazione.

Real-time Computing – Criteri

- I softmodem, che sostituiscono un modem hardware con un software che gira sulla CPU di un computer, sono sistemi real time.
  - Il software deve essere eseguito ogni pochi millisecondi per generare i prossimi dati audio da emettere.
  - Se questi dati sono in ritardo, il modem ricevente perde la sincronizzazione, causando una lunga interruzione fino a quando la sincronizzazione non viene ristabilita o causando la perdita totale della connessione.
- Molti tipi di stampanti hanno requisiti di hard real-time, come le inkjet (l'inchiostro deve essere depositato al momento giusto mentre la testina di stampa attraversa la pagina), le stampanti laser (il laser deve essere attivato al momento giusto mentre il raggio attraversa il tamburo rotante), e le stampanti a matrice di punti e vari tipi di stampanti di linea (il meccanismo di impatto deve essere attivato al momento giusto mentre il meccanismo di stampa viene allineato con l'output desiderato).
  - Un guasto in uno di questi meccanismi di stampa causerebbe un output mancante o disallineato.

- Nel contesto dei sistemi multitasking la politica di schedulazione è normalmente guidata dalla priorità (schedulatori pre-emptive).

Real-time Computing – Criteri

- In alcune situazioni, questi possono garantire prestazioni hard real-time (per esempio se l'insieme dei task e le loro priorità sono noti in anticipo).
- I sistemi firm real-time sono definiti in modo più nebuloso, e alcune classificazioni non li includono, distinguendo solo i sistemi hard e soft real-time.
- Come esempio di sistema firm real-time consideriamo la macchina della catena di montaggio descritta in precedenza per il caso hard real-time.
- Una deadline mancata provoca ancora un errore che deve essere affrontato: ci potrebbe essere un macchinario per contrassegnare un pezzo come cattivo o espellerlo dalla linea di assemblaggio, o la linea di assemblaggio potrebbe essere fermata in modo che un operatore possa correggere il problema.
- Tuttavia, finché questi errori sono poco frequenti, possono essere tollerati.
- I sistemi soft real-time sono tipicamente usati per risolvere problemi di accesso concorrente e per far fronte alla necessità di mantenere un certo numero di sistemi connessi aggiornati attraverso situazioni mutevoli.
- Alcuni esempi di sistemi soft real-time sono i seguenti:

Real-time Computing – Criteri

- Software che mantiene e aggiorna i piani di volo per gli aerei di linea commerciali.
  - I piani di volo devono essere mantenuti ragionevolmente aggiornati, ma possono operare con una latenza di pochi secondi.
- I sistemi audio-video dal vivo sono di solito anche soft real-time.
  - Un fotogramma dell'audio che viene riprodotto in ritardo può causare un breve glitch audio (e può causare che tutto l'audio successivo sia ritardato di conseguenza, causando la percezione che l'audio venga riprodotto più lentamente del normale), ma questo può essere meglio delle alternative di continuare a riprodurre silenzio, oppure di vedere un'immagine statica, oppure di sentire un audio frame precedente, o di avere dati stimati.
  - Un fotogramma di video ritardato in genere provoca ancora meno disturbo per gli spettatori.
  - Il sistema può continuare a funzionare e anche recuperare successivamente usando metodologie di previsione del carico di lavoro e riconfigurazione.

Real-time Computing – Criteri

- I videogiochi sono spesso soft real-time, in particolare perché cercano di soddisfare un frame rate prefissato.
  - Poiché l'immagine successiva non può essere calcolata in anticipo, dato che dipende dagli input del giocatore, solo un breve tempo è disponibile per eseguire tutti i calcoli necessari per generare un fotogramma del video prima che quel fotogramma debba essere visualizzato.
  - Se la scadenza non viene rispettata, il gioco può continuare a un frame rate inferiore.
  - A seconda del gioco, questo può influenzare solo la sua grafica (mentre il gameplay continua a velocità normale), o il gameplay stesso può essere rallentato (cosa comune sulle vecchie console di terza e quarta generazione).

Real-time Computing – Real-time e high-performance

- Il real-time computing è talvolta frainteso come calcolo ad alte prestazioni, ma questa non è una classificazione accurata.
- Per esempio, un enorme supercomputer che esegue una simulazione scientifica può offrire prestazioni impressionanti, ma non sta eseguendo un calcolo in tempo reale.
- Al contrario, una volta che l'hardware e il software di un sistema frenante antibloccaggio sono stati progettati per soddisfare le deadline richieste, nessun ulteriore aumento di prestazioni è obbligatorio o addirittura utile.
- Inoltre, se un server di rete è altamente caricato dal traffico di rete, il suo tempo di risposta può essere più lento, ma (nella maggior parte dei casi) riuscirà comunque ad avere successo prima di andare in time out (raggiungere la sua deadline).
- Quindi, un tale server di rete non sarebbe considerato un sistema in tempo reale: i guasti temporali (ritardi, time-out, ecc.) sono tipicamente piccoli e compartimentati (effetto limitato) ma non sono guasti catastrofici.
- In un sistema in tempo reale, come l'indice FTSE 100, un rallentamento oltre i limiti sarebbe spesso considerato catastrofico nel suo contesto applicativo.
- Il requisito più importante di un sistema in tempo reale è l'output consistente, non l’alto throughput.

Real-time Computing – Real-time e high-performance

- Alcuni tipi di software, come molti programmi per giocare a scacchi, possono rientrare in entrambe le categorie.

- Per esempio, un programma di scacchi progettato per giocare in un torneo con un clock dovrà decidere una mossa prima di una certa scadenza o perderà la partita, ed è quindi un calcolo in tempo reale, ma un programma di scacchi che può essere aspettare indefinitamente prima di muoversi non lo è.

- In entrambi i casi, comunque, le alte prestazioni sono desiderabili: più lavoro un programma di scacchi da torneo può fare nel tempo assegnato, migliori saranno le sue mosse, e più velocemente un programma di scacchi senza vincoli corre, prima sarà in grado di muovere.

- Questo esempio illustra anche la differenza essenziale tra il calcolo in real-time e gli altri tipi di calcolo: se il programma di scacchi da torneo non prende una decisione sulla sua prossima mossa nel tempo assegnato, perde la partita - cioè, fallisce come calcolo in real-time - mentre nell'altro scenario, si presume che non sia necessario rispettare la deadline.

- L'alta prestazione è indicativa della quantità di elaborazione che viene eseguita in una data quantità di tempo, mentre il tempo reale è la capacità di finire l'elaborazione per produrre un risultato utile nel tempo disponibile.

Real-time Computing – Near real-time

- Il termine «near real-time» o «nearly real-time» (NRT), nelle telecomunicazioni e nell'informatica, si riferisce al ritardo temporale introdotto, tramite l'elaborazione automatica dei dati o la trasmissione in rete, tra il verificarsi di un evento e l'utilizzo dei dati elaborati, come ad esempio per scopi di visualizzazione o feedback e controllo.
- Per esempio, una visualizzazione near real-time raffigura un evento o una situazione come esisteva al momento attuale meno il tempo di elaborazione, quasi come se fossimo in un evento dal vivo
- La distinzione tra i termini «near real-time» e «real-time» è un po' nebulosa e deve essere considerata caso per caso.
- Il termine implica che non ci sono ritardi significativi.
- In molti casi, l'elaborazione descritta come «real-time» dovrebbe essere più accuratamente denominata «near real-time».
- Near real-time si riferisce anche alla trasmissione ritardata in real-time di voce e video.
- Esso permette la riproduzione di immagini video, approssimativamente in tempo reale, senza dover aspettare il download di un intero file video di grandi dimensioni.

Real-time Computing – Near real-time

- Database tra loro incompatibili possono esportare/importare file di dati piatti comuni che l'altro database può importare/esportare secondo un opportuno scheduling in modo che possano sincronizzare/condividere dati comuni in «near real-time» l'uno con l'altro.

- La distinzione tra «near real-time» e «real-time» varia, e il ritardo preso in considerazione dipende dal tipo e dalla velocità della trasmissione.

- Il ritardo in «near real-time» è tipicamente in un intervallo di 1-10 secondi.

Cloud computing - Introduzione

- Il cloud computing è la distribuzione di servizi di calcolo, come server, risorse di archiviazione, database, rete, software, analisi e intelligence, tramite Internet ("il cloud"), per offrire innovazione rapida, risorse flessibili ed economie di scala.

- Si paga solo per i servizi cloud che si usano e si risparmia sui costi operativi, si esegue l'infrastruttura in modo più efficiente e si ridimensionano le risorse in base all'evoluzione delle esigenze aziendali.

- Il cloud computing rappresenta un grande cambiamento rispetto alla visione tradizionale delle aziende in materia di risorse IT.

- Ecco sette motivi comuni per cui le organizzazioni ricorrono ai servizi di cloud computing:

  - Costo:

    - Il cloud computing elimina le spese di capitale associate all'acquisto di hardware e software e alla configurazione e alla gestione di data center locali, che richiedono rack di server, elettricità 24 ore su 24 per alimentazione e raffreddamento ed esperti IT per la gestione dell'infrastruttura.

Cloud computing - Introduzione

- Velocità:
  - La maggior parte dei servizi di cloud computing viene fornita in modalità self-service e su richiesta, quindi è possibile effettuare il provisioning anche di grandi quantità di risorse di calcolo in pochi minuti, in genere con pochi click del mouse.
  - Le aziende possono quindi usufruire di una grande flessibilità senza la pressione legata alla pianificazione della capacità.
- Scalabilità globale:
  - I vantaggi dei servizi di cloud computing includono la possibilità di ridimensionare le risorse in modo elastico.
  - In materia di cloud questo significa fornire la giusta quantità di risorse IT, ad esempio una quantità maggiore o minore di potenza di calcolo, risorse di archiviazione e larghezza di banda, proprio quando è necessario e dalla località geografica appropriata.

Cloud computing - Introduzione

- Produttività:
  - I data center locali richiedono in genere un impegno notevole nell'organizzazione e nell'assemblaggio dei rack, che include la configurazione dell'hardware, l'applicazione di patch software e altre attività di gestione IT dispendiose in termini di tempo.
  - Il cloud computing elimina la necessità di molte di queste attività, consentendo ai team IT di dedicare il proprio tempo al raggiungimento di obiettivi aziendali più importanti.
- Prestazioni:
  - I più grandi servizi di cloud computing vengono eseguiti su una rete mondiale di data center sicuri, aggiornati regolarmente all'ultima generazione di hardware, veloce ed efficiente.
  - Questo offre diversi vantaggi rispetto a un singolo data center aziendale, tra cui latenza di rete ridotta per le applicazioni e maggiori economie di scala.

Cloud computing - Introduzione

- Affidabilità:
  - Il cloud computing aumenta la semplicità e riduce i costi di backup dei dati, ripristino di emergenza e continuità aziendale, grazie alla possibilità di eseguire il mirroring dei dati in più siti ridondanti nella rete del provider di servizi cloud.
- Sicurezza:
  - Molti provider di servizi cloud offrono un'ampia gamma di criteri, tecnologie e controlli che rafforzano il comportamento di sicurezza complessivo, grazie alla protezione di dati, app e infrastruttura dalle minacce potenziali.

Cloud computing – Tipi di cloud computing

- Non tutti i cloud sono uguali e non sempre lo stesso tipo di cloud computing è adatto a tutte le esigenze.
- Sono disponibili numerosi modelli, tipi e servizi diversi per offrire la soluzione più adatta in base alle esigenze.
- Per prima cosa, dobbiamo determinare il tipo di distribuzione cloud, ovvero l'architettura di cloud computing, in cui verranno implementati i servizi cloud.
- Ci sono tre modalità diverse di distribuzione dei servizi cloud: in un cloud pubblico, in un cloud privato e in un cloud ibrido.
  - Cloud pubblico:
    - I cloud pubblici sono di proprietà di un provider di servizi cloud di terze parti, che fornisce le risorse di calcolo, come server e risorse di archiviazione, tramite Internet.
    - Microsoft Azure è un esempio di cloud pubblico.
    - In un cloud pubblico, l'hardware, il software e l'infrastruttura di supporto appartengono al provider di servizi cloud, che li gestisce.
    - Si può accedere a questi servizi e gestire il proprio account usando un Web browser.

Cloud computing – Tipi di cloud computing

- Cloud privato:
  - Un cloud privato si riferisce alle risorse di cloud computing usate esclusivamente da una singola azienda o organizzazione.
  - Un cloud privato può trovarsi fisicamente nel data center locale della società.
  - Alcune società, inoltre, pagano provider di servizi di terze parti per ospitare il proprio cloud privato.
  - Un cloud privato è un cloud in cui servizi e infrastruttura sono gestiti in una rete privata.
- Cloud ibrido:
  - I cloud ibridi combinano cloud privato e pubblico, grazie a una tecnologia che consente la condivisione di dati e applicazioni tra i due tipi di cloud.
  - Grazie alla possibilità di spostare dati e applicazioni tra cloud pubblici e privati, un cloud ibrido offre all'azienda maggiore flessibilità e più opzioni di distribuzione e aiuta a ottimizzare l'infrastruttura esistente, la sicurezza e la conformità.

Cloud computing – Tipi di servizi cloud

- La maggior parte dei servizi di cloud computing rientra in quattro ampie categorie: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), elaborazione serverless e Software as a Service (SaaS).

- Talvolta si parla di "stack" di cloud computing, in quanto queste categorie sono basate una sull'altra.

  - Infrastructure as a Service:

    - Si tratta della categoria di base dei servizi di cloud computing.

    - Con una soluzione IaaS, si affitta l'infrastruttura IT, ovvero server e macchine virtuali (VM), risorse di archiviazione, reti e sistemi operativi, da un provider di servizi cloud con pagamento in base al consumo.

  - Platform as a Service:

    - Si riferisce a servizi di cloud computing che forniscono un ambiente su richiesta per lo sviluppo, il test, la distribuzione e la gestione di applicazioni software.

    - Una soluzione PaaS è progettata per consentire agli sviluppatori di creare in modo più semplice e rapido app Web o per dispositivi mobili, senza doversi preoccupare della configurazione o della gestione dell'infrastruttura di server sottostante, della rete di archiviazione e dei database necessari per lo sviluppo.

Cloud computing – Tipi di servizi cloud

- Elaborazione serverless:
  - In sovrapposizione con il modello PaaS, l'elaborazione serverless si concentra sulla funzionalità di creazione delle app senza che sia necessario spendere tempo per la gestione dei server e dell'infrastruttura necessari.
  - Il provider di servizi cloud gestisce automaticamente la configurazione, la pianificazione della capacità e la gestione dei server.
  - Le architetture serverless sono basate su eventi, offrono scalabilità elevata e usano le risorse solo quando si verifica una funzione o un trigger specifico.
- Software as a Service:
  - È un metodo per la distribuzione di applicazioni software tramite Internet, su richiesta e in genere in base a una sottoscrizione.
  - Con una soluzione SaaS, i provider di servizi cloud ospitano e gestiscono l'applicazione software e l'infrastruttura sottostante e si occupano delle attività di manutenzione, come gli aggiornamenti software e l'applicazione di patch di protezione.

Cloud computing – Tipi di servizi cloud

- Gli utenti si connettono all'applicazione tramite Internet, in genere con un Web browser nel telefono, tablet o PC.

Cloud computing – Usi del cloud computing

- Probabilmente stiamo usando il cloud computing ogni giorno senza renderci conto.
- Se usiamo un servizio online per inviare posta elettronica, modificare documenti, guardare film o programmi TV, ascoltare musica, giocare oppure archiviare immagini o altri file, è probabile che tutto questo sia possibile grazie al cloud computing, che agisce dietro le quinte.
- I primi servizi di cloud computing risalgono appena a una decina di anni fa, ma già molte organizzazioni, dalle piccole startup alle multinazionali, dagli enti pubblici alle organizzazioni no profit, stanno adottando questa tecnologia per i motivi più vari.
- Ecco alcuni esempi di ciò che è possibile fare oggi con i servizi cloud di un provider di servizi cloud:
  - Creazione di applicazioni native del cloud:
    - È possibile creare, distribuire e ridimensionare rapidamente le applicazioni per il Web, i dispositivi mobili e le API.
    - A tal fine si possono sfruttare i vantaggi delle tecnologie e degli approcci nativi del cloud, tra cui container, Kubernetes, architettura basata su microservizi, comunicazioni basate su API e DevOps.

Cloud computing – Usi del cloud computing

- Testare e compilare le applicazioni:
  - È possibile ridurre i costi e i tempi di sviluppo delle applicazioni usando infrastrutture cloud che consentono di aumentare o ridurre facilmente le prestazioni in base alle esigenze.
- Archiviare i dati ed eseguirne il backup e il ripristino:
  - È possibile proteggere i dati razionalizzando i costi e su vasta scala, grazie alla possibilità di trasferire i dati tramite Internet su un sistema di archiviazione cloud esterno accessibile da qualsiasi posizione e da qualunque dispositivo.
- Analizzare i dati:
  - È possibile unificare i dati tra team, divisioni e sedi nel cloud.
  - A questo punto è possibile usare i servizi cloud, come Machine Learning e intelligenza artificiale, per acquisire informazioni dettagliate e prendere decisioni più informate.

Cloud computing – Usi del cloud computing

- Trasmettere in streaming audio e video:
  - È possibile rimanere in contatto con i propri destinatari ovunque, in qualsiasi momento e su qualunque dispositivo, grazie alle funzionalità audio e video ad alta definizione con distribuzione globale.
- Incorporare l'intelligence:
  - È possibile usare modelli intelligenti per coinvolgere i clienti e raccogliere informazioni dettagliate preziose dai dati acquisiti.
- Fornire software on demand:
  - Tramite l'utilizzo del paradigma Software as a Service, possiamo fornire le versioni e gli aggiornamenti più recenti del software ai nostri clienti, sempre e ovunque si trovino.

L'architettura a microservizi - Introduzione

- I microservizi sono una metodologia di sviluppo di applicazioni basata sui servizi.
- In essa, le grandi applicazioni vengono divise in unità di servizio indipendenti più piccole.
- I microservizi rappresentano un metodo di implementazione della Service Oriented Architecture (SOA) che prevede di dividere l'intera applicazione in una collezione di servizi interconnessi, dove ogni servizio soddisferà solo un bisogno di business.
- In una Service Oriented Architecture, interi pacchetti software sono suddivisi in piccole unità di business interconnesse.
- Ognuna di queste piccole unità di business comunicherà con le altre usando diversi protocolli per consegnare al cliente un business di successo.
- Ora la domanda è: in che modo la Microservice Architecture (MSA) differisce dalla SOA?
- La risposta è semplice: SOA è un pattern di progettazione e i microservizi rappresentano una metodologia di implementazione per realizzare la SOA o, in altre parole, i microservizi possono essere visti come un caso particolare di SOA.

L’architettura a microservizi - Introduzione

- Di seguito ci sono alcune regole che dobbiamo tenere a mente durante lo sviluppo di un'applicazione orientata ai microservizi:
  - Indipendente: ogni microservizio dovrebbe essere distribuibile in modo indipendente.
  - Accoppiamento: tutti i microservizi dovrebbero essere lascamenti accoppiati l'uno con l'altro in modo che i cambiamenti in uno non influenzino l'altro.
  - Obiettivo di business: ogni unità di servizio dell'intera applicazione dovrebbe essere la più piccola e capace di fornire uno specifico obiettivo di business.
- Consideriamo un esempio di portale di shopping online per capire a fondo i microservizi.
- Ora, suddividiamo l'intero portale di e-commerce in piccole unità di business come la gestione degli utenti, la gestione degli ordini, il check-in, la gestione dei pagamenti, la gestione delle consegne, ecc.
- Un ordine di successo deve procedere attraverso tutti questi moduli in un lasso di tempo specifico.

L'architettura a microservizi - Introduzione

- Nella seguente figura viene mostrata un'immagine delle diverse unità di business associate al sistema di commercio elettronico:



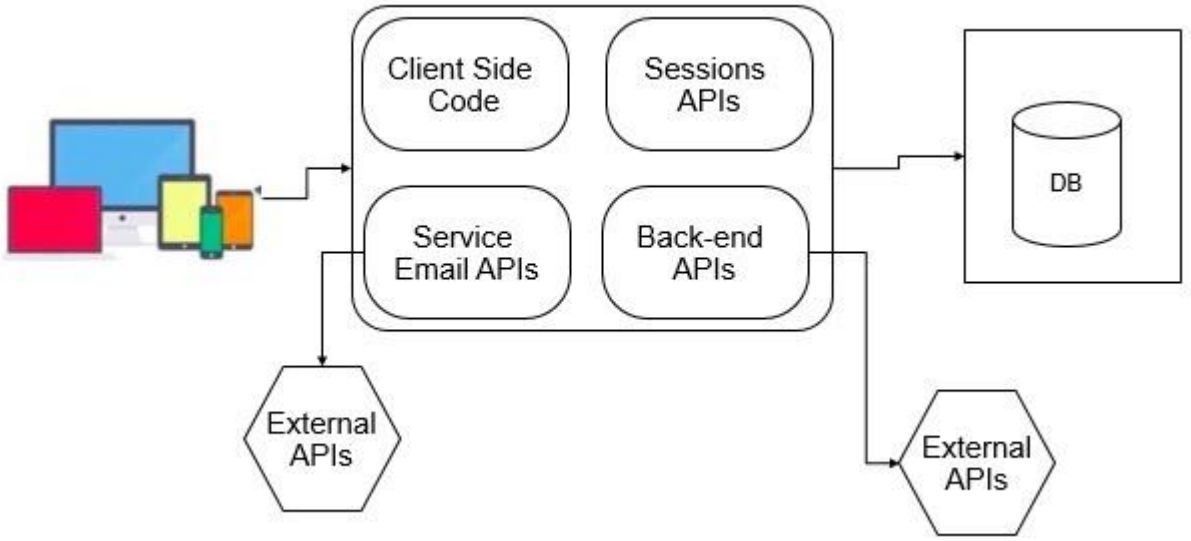
- Ognuno di questi moduli di business dovrebbe avere la propria logica di business e i propri stakeholder.
- Essi comunicano con altri software di terze parti per alcune esigenze specifiche, e anche tra di loro.
- Per esempio, la gestione degli ordini può comunicare con la gestione degli utenti per ottenere informazioni sugli utenti.

L'architettura a microservizi - Introduzione

- Ora, considerando che stiamo gestendo un portale di shopping online con tutte queste unità di business menzionate prima, abbiamo bisogno di un'applicazione di livello enterprise composta da diversi layer come front-end, back-end, database, ecc.

- Se la nostra applicazione non è scalabile ed è sviluppata in un unico file war, allora sarà una tipica applicazione monolitica.

- Secondo IBM, una tipica applicazione monolitica dovrebbe possedere internamente la seguente struttura di moduli dove un solo endpoint o applicazione sarà responsabile di gestire tutte le richieste degli utenti.

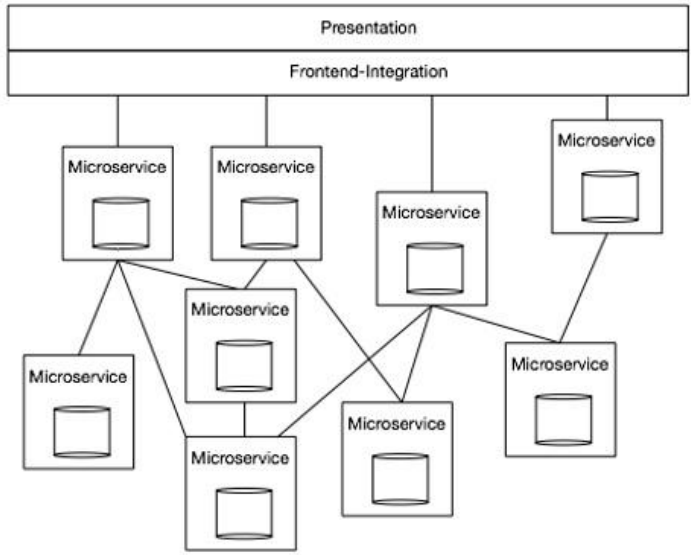


L’architettura a microservizi - Introduzione

- Nell'immagine precedente, possiamo vedere diversi moduli come il database per memorizzare diversi utenti e dati aziendali.
- Al front-end abbiamo diversi dispositivi dove di solito vengono forniti i dati degli utenti o delle imprese da utilizzare.
- Nel mezzo, abbiamo un pacchetto che può essere un file EAR o WAR distribuibile che accetta la richiesta da parte degli utenti, la elabora con l'aiuto delle risorse e fornisce le risposte agli utenti.
- Tutto andrà bene fino a quando il business non vorrà dei cambiamenti nell’applicazione precedente.
- Consideriamo i seguenti scenari in cui dobbiamo cambiare la nostra applicazione in base alle esigenze di business.
- La business unit ha bisogno di alcune modifiche nel modulo "Ricerca".
- Quindi, è necessario cambiare l'intero processo di ricerca ed effettuare di nuovo il deployment dell’applicazione.
- In questo caso, dovremo rifare il deployment di tutte le altre unità che non hanno subito alcun cambiamento.
- Supponiamo ora che ci vengano richiesti alcuni cambiamenti nel modulo "Check out" per includere l'opzione "portafoglio".

L’architettura a microservizi - Introduzione

- Ora dobbiamo cambiare il nostro modulo "Check out" ed effettuare di nuovo il deployment dello stesso nel server.
- Ancora una volta dovremo fare il deployment di vari moduli contenuti nei vari software package senza aver apportato ad essi alcuna modifica.
- Qui può entrare in gioco l’implementazione tramite microservizi della Service Oriented Architecture.
- Possiamo sviluppare la nostra applicazione monolitica in modo tale che ogni modulo del software si comporti come un'unità indipendente, capace di gestire un singolo task di business in modo indipendente.



L’architettura a microservizi - Introduzione

- Nell'architettura precedente non stiamo creando alcun file compatto e monolitico.
- Invece, stiamo dividendo diverse parti del software esponendole come servizio.
- Ogni parte del software può facilmente comunicare con l'altra consumando i rispettivi servizi.
- Ecco come il microservizio gioca un grande ruolo nelle moderne applicazioni web.
- Vediamo come sarebbe la gestione del carrello della spesa in un’architettura a microservizi.
- Possiamo suddividere il nostro sistema di e-commerce in diversi moduli come "Ricerca", "Filtro", "Cassa", "Carrello", "Raccomandazione", ecc.
- Se vogliamo costruire un portale di e-commerce, allora dobbiamo costruire i moduli di cui sopra in modo tale che possano connettersi tra loro per darci una buona esperienza di shopping 24x7.

L'architettura a microservizi - Introduzione

- Supponiamo, ad esempio, di voler costruire un'applicazione per la prenotazione dei taxi tramite la Service Oriented Architecture.
- Le sue unità di software saranno:
  - GetPayments And DriverInformation And MappingData API
  - AuthenticateUsers And Drivers API
- Se la stessa applicazione viene costruita attraverso un'architettura a microservizi, le sue API saranno:
  - SubmitPaymentsService
  - GetDriverInfoService
  - GetMappingDataService
  - AuthenticateUserService
  - AuthenticateDriverService

L'architettura a microservizi – Vantaggi

- I vantaggi di un'architettura a microservizi rispetto ad un'architettura monolitica sono i seguenti:
  - Piccole dimensioni:
    - I microservizi sono un'implementazione della Service Oriented Architecture.
    - Si raccomanda di mantenere il servizio il più piccolo possibile.
    - Fondamentalmente, un servizio non dovrebbe eseguire più di un business task, quindi sarà ovviamente di piccole dimensioni e facile da mantenere rispetto a qualsiasi altra applicazione monolitica.
  - Focalizzazione:
    - Ogni microservizio è progettato per svolgere un solo business task.
    - Durante la progettazione di un microservizio, l'architetto dovrebbe preoccuparsi del punto focale del servizio, che è il suo output.
    - Per definizione, un microservizio dovrebbe essere full stack in natura e dovrebbe essere impegnato ad effettuare la consegna di uno ed un solo output

L’architettura a microservizi – Vantaggi

- Autonomia:
  - Ogni microservizio dovrebbe essere un'unità di business autonoma dell'intera applicazione.
  - Quindi, l'applicazione diventa più lascamente accoppiata, il che aiuta a ridurre il costo di manutenzione.
- Eterogeneità della tecnologia:
  - I microservizi supportano diverse tecnologie per far comunicare tra loro le varie unità di business, il che aiuta gli sviluppatori a usare la tecnologia corretta al posto giusto.
  - Implementando un sistema eterogeneo, si può ottenere la massima sicurezza, velocità e un sistema scalabile.
- Resilienza:
  - La resilienza è una proprietà di isolamento di un'unità software.
  - L’architettura a microservizi segue un alto livello di resilienza nella metodologia di costruzione, quindi ogni volta che un'unità fallisce non ha impatto sull'intero business.

L’architettura a microservizi – Vantaggi

- La resilienza è un'altra proprietà che caratterizza un sistema altamente scalabile e meno accoppiato.

- Facilità di deployment:

  - Poiché l'intera applicazione è suddivisa in piccoli pezzi di unità, ogni componente dovrebbe essere full stack in natura.

  - Per ciascuno di loro si può effettuare il deployment in qualsiasi ambiente molto facilmente con meno complessità di tempo a differenza di altre applicazioni monolitiche dello stesso tipo.

L’architettura a microservizi – Svantaggi

- Un’architettura a microservizi ha anche però degli svantaggi, i principali dei quali sono i seguenti:
  - Sistema distribuito:
    - A causa dell'eterogeneità tecnica, saranno utilizzate diverse tecnologie per sviluppare diverse parti di un microservizio.
    - Un enorme insieme di professionisti qualificati è necessario per supportare questo grande software distribuito eterogeneo.
    - Quindi, la distribuzione e l'eterogeneità sono uno svantaggio importante dell'uso dei microservizi.
  - Costo:
    - L’architettura a microservizi è costosa, in quanto è necessario mantenere uno spazio server diverso per diversi task aziendali.

L’architettura a microservizi – Svantaggi

- Prontezza per l’impresa:
  - L'architettura a microservizi può essere considerata come un conglomerato di diverse tecnologie, poiché la tecnologia si evolve giorno per giorno.
  - Quindi, è abbastanza difficile rendere un'applicazione a microservizi pronta per l'impresa rispetto al modello di sviluppo software convenzionale.

L’architettura a microservizi – Scaling

- Lo scaling è un processo di suddivisione di un software in diverse unità.
- Lo scaling si definisce anche in termini di scalabilità.
- La scalabilità è il potenziale per implementare caratteristiche più avanzate dell'applicazione.
- Aiuta a migliorare la sicurezza, la durata e la manutenibilità dell'applicazione.
- Abbiamo tre tipi di procedure di scaling che vengono seguite nelle industrie.
- Di seguito ci sono le diverse metodologie di scalabilità con i corrispondenti esempi di vita reale.
  - Scaling sull’asse delle X:
    - Lo scaling sull’asse delle X è anche chiamato scaling orizzontale.
    - In questa procedura, l'intera applicazione è suddivisa in diverse parti orizzontali.
    - Normalmente, qualsiasi applicazione web server può avere questo tipo di scaling.

L’architettura a microservizi – Scaling

- Scaling sull’asse delle Y:
  - Lo scaling dell'asse Y è anche chiamato scaling verticale e comporta lo scaling di tutti i livelli delle risorse.
  - In questo tipo di scaling, la richiesta degli utenti viene reindirizzata e limitata implementando una certa logica.
  - Consideriamo Facebook come esempio. Facebook ha bisogno di gestire 1,79 milioni di utenti ogni secondo; quindi, controllare il traffico è una grande responsabilità degli ingegneri di rete di Facebook.
  - A tal fine essi eseguono lo scaling sull’asse delle Y che include l'esecuzione di più server con la stessa applicazione allo stesso tempo.

L’architettura a microservizi – Scaling

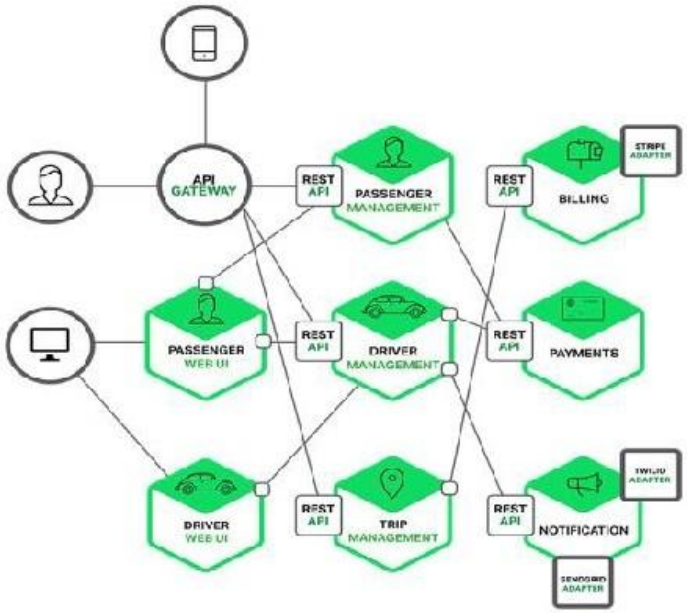
- Ora, al fine di controllare questo enorme livello di traffico, Facebook reindirizza tutto il traffico di una regione a un server specifico, come illustrato nella figura sottostante.

- Questo trasferimento di traffico basato sulla regione è chiamato bilanciamento del carico nel linguaggio architetturale.



L’architettura a microservizi – Scaling

- Scaling sull’asse delle Z:
  - Lo scaling degli assi X e Y è molto più facile da capire.
  - Tuttavia, un'applicazione può anche essere scalata a livello di business, il che è chiamato scaling sull'asse Z.
  - Di seguito è riportato un esempio di scaling di un'applicazione di servizio taxi nelle diverse verticali di unità di business.



L’architettura a microservizi – Modello

- I seguenti sono i principi che devono essere presi in considerazione durante lo sviluppo di un microservizio:
  - Alta coesione: tutti i modelli di business devono essere suddivisi nella parte di business più piccola possibile; ogni servizio dovrebbe essere focalizzato sull'esecuzione di un solo compito aziendale.
  - Indipendenza: tutti i servizi dovrebbero essere full stack in natura e indipendenti l'uno dall'altro.
  - Centrato sul dominio di business: la suddivisione del software è in genere effettuata rispetto all’unità di business e non in base ai livelli.
  - Automazione: la gestione dei test sarà automatizzata e si cerca di minimizzare l’interazione umana.
  - Osservabile - Ogni servizio sarà full stack in natura e dovrebbe poter essere distribuito indipendentemente dagli altri e si dovrebbe poter trattare come un’applicazione aziendale completa.
- "Two Pizza Rule" è un tipo di regola che limita il numero di partecipanti in un team di sviluppo di microservizi.
- Secondo questa regola, il numero dei membri del team di un'applicazione dovrebbe essere così piccolo da poter essere nutrito da due pizze.

L’architettura a microservizi – Modello

- Generalmente, il numero non dovrebbe essere più di 8.
- Poiché il microservizio è full stack in natura, anche il team è full stack in natura.
- Per aumentare la produttività, dobbiamo costruire un team di massimo 8 membri con tutti i tipi di competenze richieste per quel servizio.
- Il task è un ruolo importante nel ciclo di vita dello sviluppo del software.
- Lo sviluppo di un'applicazione su larga scala può essere suddiviso in diverse piccole unità di task.
- Consideriamo di dover sviluppare un'applicazione come Facebook. Allora, la funzionalità "Log in" può essere considerata come un task dell'intero processo di costruzione.
- I progressi di ciascuno di questi task devono essere monitorati opportunamente da professionisti altamente qualificati.
- Questo modo di procedere si sposa perfettamente con i metodi agile di ingegneria del software.

L'architettura a microservizi – Sicurezza

- Quando si tratta di trattare tonnellate di dati dei clienti, la sicurezza gioca un ruolo importante.
- Il problema della sicurezza è associato a tutti i tipi di servizi disponibili sul mercato.
- Qualunque sia il cloud che si sta utilizzando - privato, pubblico, ibrido, etc., la sicurezza dovrebbe essere mantenuta a tutti i livelli.
- Il problema complessivo della sicurezza può essere sostanzialmente suddiviso nelle seguenti parti:
  - Problema di sicurezza affrontato dai fornitori di servizi:
    - Questo tipo di problema di sicurezza è affrontato dai fornitori di servizi come Google, Amazon, etc.
    - Per garantire la protezione della sicurezza, il controllo in background del cliente è necessario soprattutto per coloro che hanno accesso diretto alla parte centrale del cloud.

L'architettura a microservizi – Sicurezza

- Problema di sicurezza affrontato dai consumatori:
  - Alcune organizzazioni memorizzano i dettagli degli utenti in data center di terze parti, e tirano i dati ogni volta che è necessario.
  - Quindi, è obbligatorio mantenere livelli di sicurezza tali che qualsiasi dato privato di un cliente non dovrebbe essere visibile a nessun altro utente.

- Per prevenire i problemi di sicurezza di cui sopra, ecco alcuni dei meccanismi difensivi utilizzati dalle organizzazioni.
  - Controllo deterrente: conoscere la potenziale minaccia per ridurre gli attacchi informatici.
  - Controllo preventivo: mantenere una politica di autenticazione di alto livello per accedere al cloud.
  - Controllo investigativo: monitorare gli utenti e rilevare qualsiasi rischio potenziale.
  - Controllo correttivo: lavorare a stretto contatto con diversi team e risolvere i problemi che sorgono durante la fase di controllo investigativo.

L’architettura a microservizi – Pattern di composizione

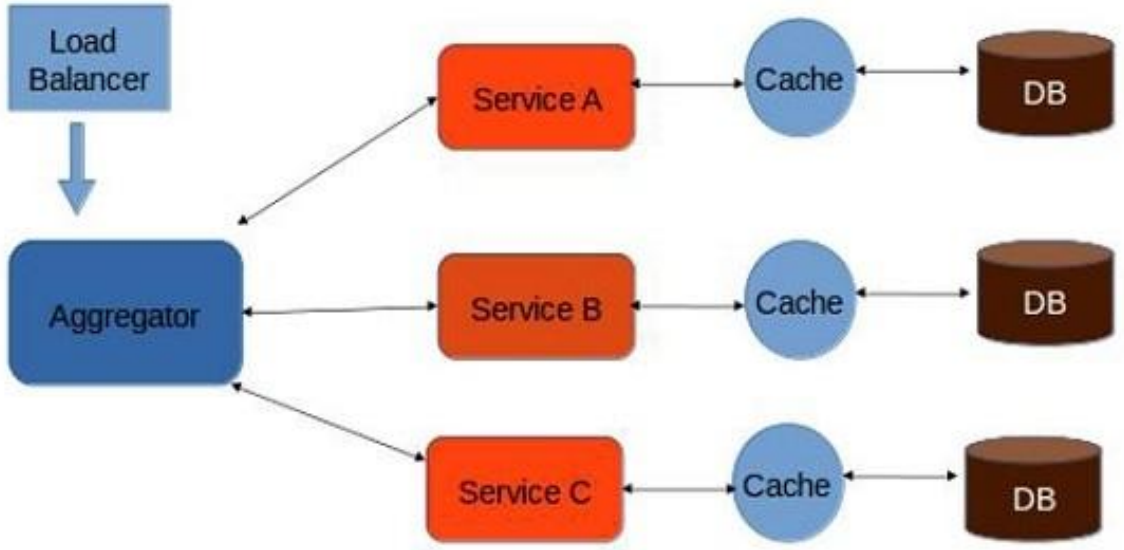
- La composizione del software indica il modo con cui costruire un prodotto software a partire da componenti più piccoli.

- Fondamentalmente si tratta di definire un’architettura software di alto livello in cui diversi moduli del nostro software comunicheranno per specifici obiettivi di business.

- La decomposizione funzionale gioca un ruolo importante nella costruzione dei microservizi in quanto fornisce agilità, flessibilità e scalabilità alla nostra applicazione.

L'architettura a microservizi – Pattern di composizione – Pattern Aggregator

- Il pattern Aggregator è il più semplice pattern web che può essere implementato durante lo sviluppo di un microservizio.
- In questo pattern di composizione, un semplice modulo web agirà come un bilanciatore di carico, il che significa che chiamerà diversi servizi secondo i requisiti.
- Di seguito è riportato un diagramma che rappresenta una semplice web app a microservizi che utilizza il pattern aggregator.

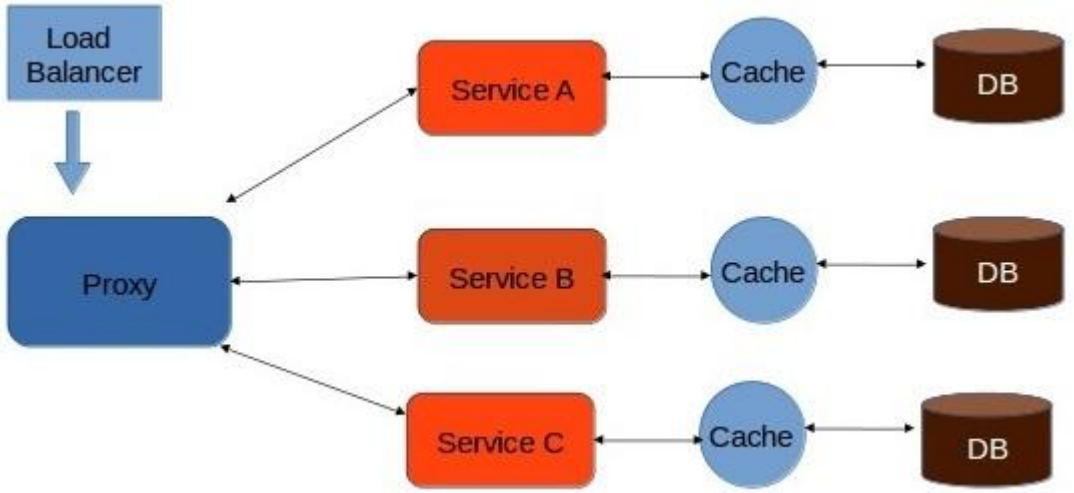


L’architettura a microservizi – Pattern di composizione – Pattern Aggregator

- Come si vede nell'immagine, l'"aggregatore" è responsabile della chiamata di diversi servizi uno per uno.
- Se abbiamo bisogno di applicare qualsiasi logica di business sui risultati del servizio A, B e C, allora possiamo implementare la logica di business nell'aggregatore stesso.
- Un aggregatore può essere a sua volta esposto come un altro servizio al mondo esterno, che può essere consumato da altri quando richiesto.
- Mentre sviluppiamo il servizio web con il modello dell'aggregatore, dobbiamo tenere a mente che ognuno dei nostri servizi A, B e C dovrebbe avere i propri strati di caching e dovrebbe essere full stack in natura.

L'architettura a microservizi – Pattern di composizione – Proxy Pattern

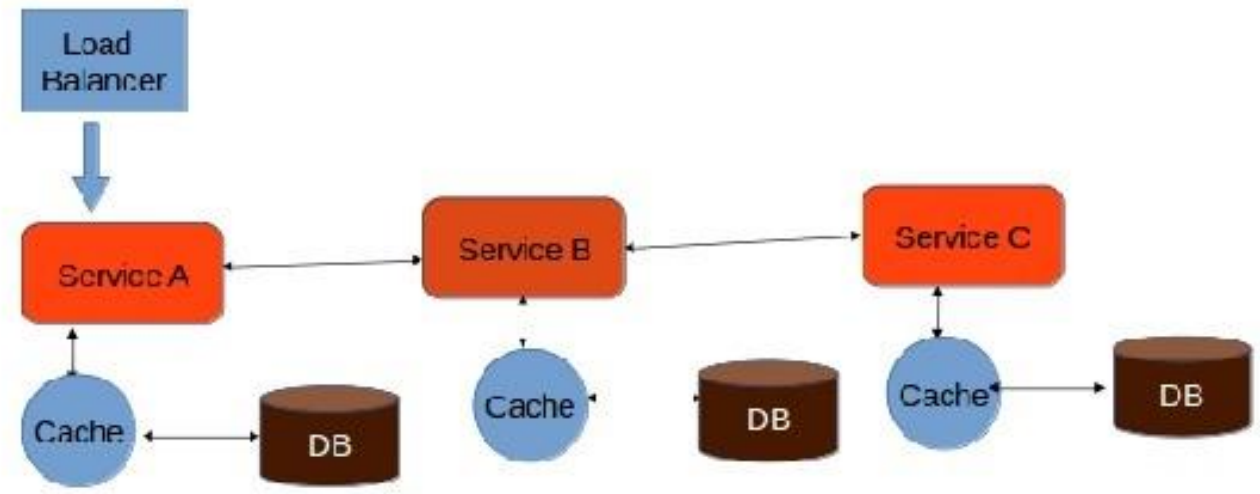
- Il proxy microservice pattern è una variante del pattern aggregator.
- In questo pattern useremo un modulo proxy invece di un modulo di aggregazione.
- Il servizio proxy può chiamare diversi servizi singolarmente.



- Nel pattern Proxy, possiamo costruire un livello di sicurezza in più fornendo un livello proxy dump.
- Questo livello agisce in modo simile ad un'interfaccia (nel senso object-oriented del termine).

L'architettura a microservizi – Pattern di composizione – Chained Pattern

- Come suggerisce il nome, questo tipo di pattern di composizione seguirà la struttura a catena.
- Qui, non useremo nulla tra il client e il livello dei servizi.
- Invece, permetteremo al client di comunicare direttamente con i servizi e tutti i servizi saranno concatenati in modo tale che l'output di un servizio sarà l'input del servizio successivo.
- La seguente figura mostra un tipico microservizio a catena.



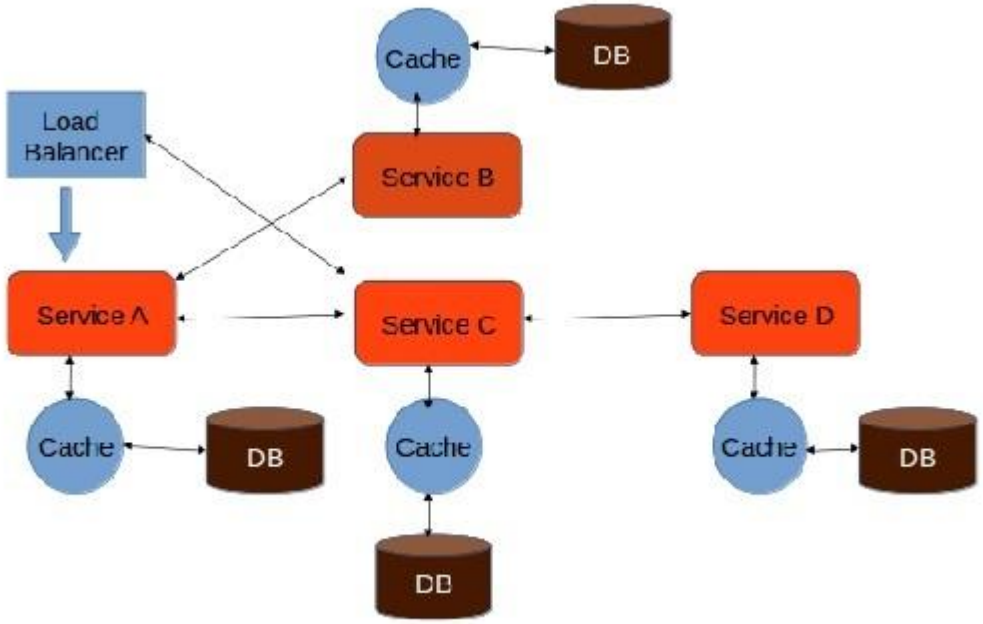
L’architettura a microservizi – Pattern di composizione – Chained Pattern

- Uno degli svantaggi principali di questa architettura è che il client sarà bloccato fino a quando l'intero processo non sarà completato.

- Quindi, è altamente raccomandabile mantenere la lunghezza della catena la più breve possibile.

L'architettura a microservizi – Pattern di composizione – Branch Microservice Pattern

- Il Branch Microservice pattern è la versione estesa dell'Aggregator pattern e del Chain pattern.
- In questo design pattern, il cliente può comunicare direttamente con il servizio.
- Inoltre, un servizio può comunicare con più di un servizio alla volta.
- Nella seguente figura è riportato un diagramma che mostra il funzionamento di questo pattern.

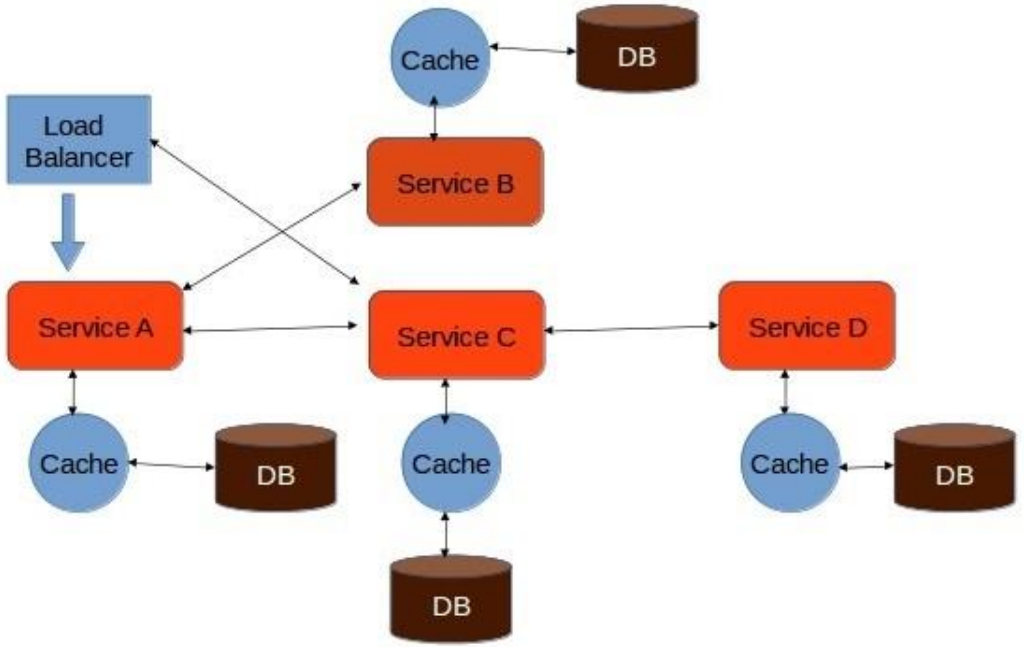


L'architettura a microservizi – Pattern di composizione – Branch Microservice Pattern

- Il Branch Microservice Pattern permette allo sviluppatore di configurare le chiamate dei servizi in modo dinamico.
- Tutte le chiamate di servizio avverranno in modo concorrente, il che significa che il servizio A può chiamare il servizio B e C contemporaneamente.

L'architettura a microservizi – Pattern di composizione – Shared Resource Pattern

- Lo Shared Resource Pattern è in realtà un conglomerato di tutti i tipi di pattern menzionati in precedenza.
- In questo pattern, il cliente o il bilanciatore di carico comunicherà direttamente con ciascun servizio quando necessario.
- Questo è il pattern di progettazione più efficace seguito ampiamente nella maggior parte delle organizzazioni.
- Di seguito è riportato un diagramma di funzionamento di questo pattern.



Dependability dei sistemi software - Introduzione

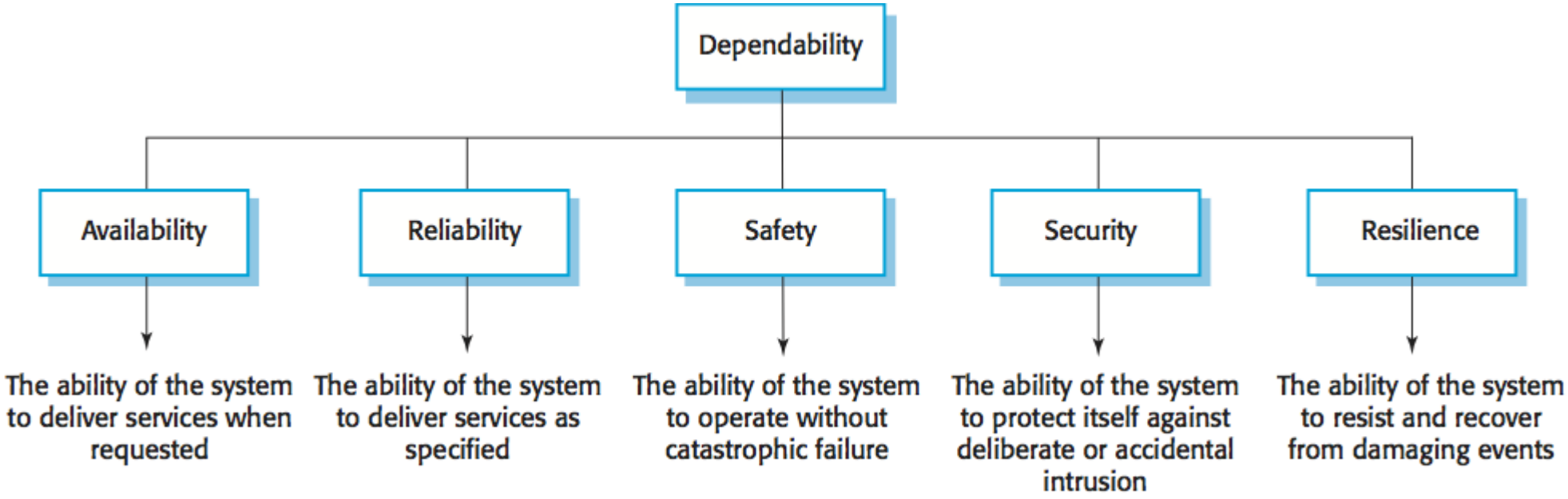
- Per molti sistemi software, la proprietà più importante è la dependability del sistema.
- La dependability di un sistema riflette il grado di fiducia dell'utente in quel sistema.
- Essa riflette il grado di fiducia che l'utente ripone nel fatto che il sistema si comporterà come egli si aspetta e che non "fallirà" durante il normale utilizzo.
- La dependability tiene conto delle proprietà di reliability, availabilty e security.
- Questa tre proprietà sono tra loro correlate.
- I guasti del sistema possono avere effetti diffusi con un gran numero di persone colpite dal guasto.
- I sistemi che non rispettano la dependability, e quindi le tre proprietà di reliability, availabilty e security, possono essere rifiutati dai loro utenti.
- I costi del fallimento del sistema possono essere molto alti se il fallimento porta a perdite economiche o danni fisici.
- I sistemi che non rispettano la dependability possono causare la perdita di informazioni con un alto costo di recupero conseguente.

Dependability dei sistemi software - Introduzione

- Le cause del fallimento possono essere di tre tipi:
  - Guasto dell'hardware:
    - L'hardware si guasta a causa di errori di progettazione e fabbricazione o perché i componenti hanno raggiunto la fine della loro vita naturale.
  - Guasto del software:
    - Il software si guasta a causa di errori nella sua specificazione, progettazione o implementazione.
  - Guasto operativo:
    - Gli operatori umani commettono errori; questa è forse la causa maggiore di fallimento nei sistemi socio-tecnici.

Dependability dei sistemi software – Principali proprietà della dependability

- Nella seguente figura vengono elencate le principali proprietà della dependability:



Dependability dei sistemi software – Principali proprietà della dependability

- Le principali proprietà della dependability sono le seguenti:
  - Availability:
    - La probabilità che il sistema sia attivo e funzionante e in grado di fornire servizi utili agli utenti.
  - Reliability:
    - La probabilità che il sistema fornisca correttamente i servizi che gli utenti si aspettano.
  - Safety:
    - Un giudizio sulla probabilità che il sistema non causi danni alle persone o al suo ambiente.
  - Security:
    - Un giudizio sulla probabilità che il sistema possa resistere a intrusioni accidentali o deliberate.

Dependability dei sistemi software – Principali proprietà della dependability

- Resilience:
  - Un giudizio su come un sistema può mantenere la continuità dei suoi servizi critici in presenza di eventi dirompenti come guasti alle apparecchiature e attacchi informatici.

Dependability dei sistemi software – Principali proprietà della dependability

- Altre proprietà della dependability di un sistema software sono le seguenti:
  - Repairability:
    - Riflette la misura in cui il sistema può essere riparato in caso di guasto.
  - Maintainability:
    - Riflette la misura in cui il sistema può essere adattato a nuovi requisiti.
  - Survivability:
    - Riflette la misura in cui il sistema può fornire servizi mentre è sotto attacco ostile.
  - Error tolerance:
    - Riflette la misura in cui gli errori di input dell'utente possono essere evitati e tollerati.

Dependability dei sistemi software – Principali proprietà della dependability

- Molti attributi della dependability dipendono l'uno dall'altro.
- Il funzionamento sicuro (safe) del sistema dipende dal fatto che il sistema sia disponibile (available) e funzioni in modo affidabile (reliable).
- Un sistema può essere inaffidabile (unreliable) perché i suoi dati sono stati corrotti da un attacco esterno.
- Gli attacchi di denial of service su un sistema hanno lo scopo di renderlo non disponibile (unavailable).
- Se un sistema è infettato da un virus, non si può essere sicuri della sua affidabilità (reliability) o sicurezza (safety).

Dependability dei sistemi software – Come ottenere la dependability

- Per ottenere la dependability è opportuno osservare i seguenti accorgimenti:
  - Evitare l'introduzione di errori accidentali durante lo sviluppo del sistema.
  - Progettare processi di verifica e validazione efficaci per scoprire errori residui nel sistema.
  - Progettare sistemi che siano tolleranti ai guasti, in modo che possano continuare a funzionare quando si verificano i guasti.
  - Progettare meccanismi di protezione che proteggano da attacchi esterni.
  - Configurare il sistema correttamente per il suo ambiente operativo.
  - Includere funzionalità nel sistema che lo rendano capace di riconoscere i cyberattacchi e di resistere ad essi.
  - Includere meccanismi di recovery per aiutare a ripristinare il normale servizio del sistema dopo un guasto.

Dependability dei sistemi software – Come ottenere la dependability

- I costi di dependability tendono ad aumentare esponenzialmente quando sono richiesti livelli crescenti di dependability, e ciò a causa di due motivi, ovvero:
  - L'uso di tecniche di sviluppo e di hardware più costosi che sono necessari per raggiungere i livelli più alti di dependability.
  - L'aumento dei test e della convalida del sistema che è necessario per convincere il cliente del sistema e i regolatori che i livelli richiesti di dependability sono stati raggiunti.

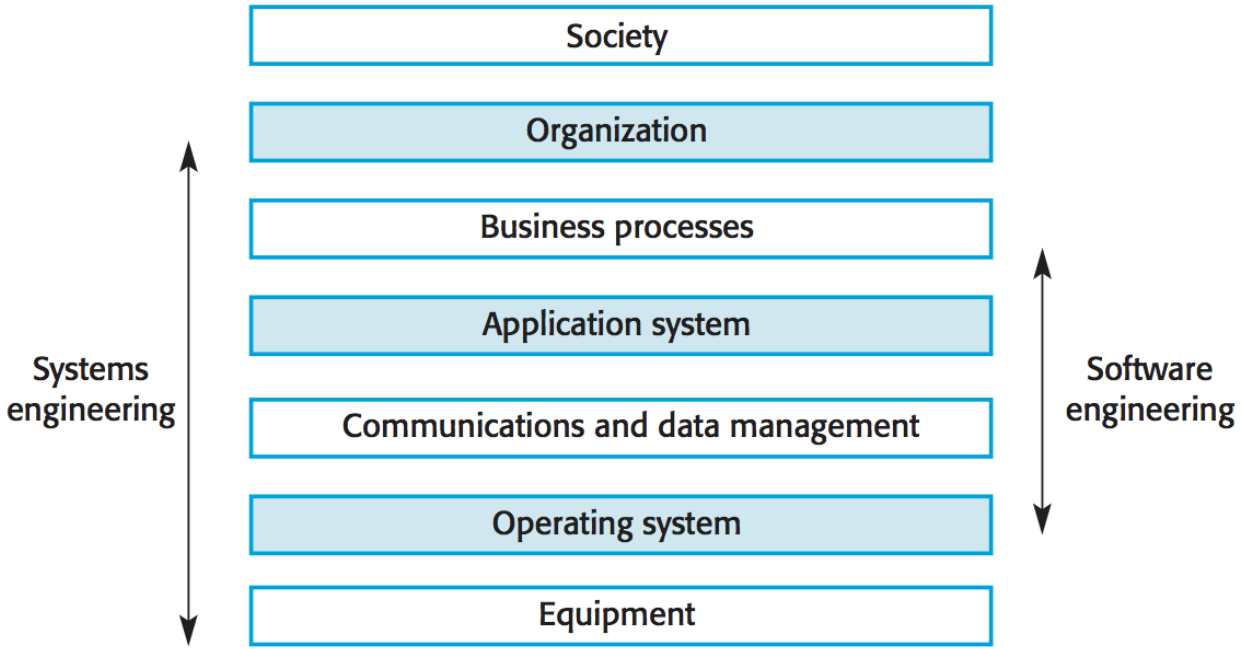
Dependability dei sistemi software – Sistemi socio-tecnici

- L'ingegneria del software non è un'attività isolata ma fa parte di un più ampio processo di ingegneria dei sistemi.
- I sistemi software non sono quindi sistemi isolati ma sono componenti essenziali di sistemi più ampi che hanno uno scopo umano, sociale o organizzativo.
- In tali sistemi è possibile riconoscere i seguenti componenti:
  - Apparecchiatura: dispositivi hardware, alcuni dei quali possono essere computer; la maggior parte dei dispositivi includerà un sistema incorporato di qualche tipo.
  - Sistema operativo: fornisce un insieme di strutture comuni per i livelli superiori del sistema.
  - Comunicazioni e gestione dei dati: middleware che fornisce l'accesso a sistemi e database remoti.
  - Sistemi applicativi: funzionalità specifiche per soddisfare alcuni requisiti dell'organizzazione.
  - Processi di business: un insieme di processi che coinvolgono persone e sistemi informatici che supportano le attività del business.
  - Organizzazioni: attività aziendali strategiche di livello superiore che influenzano il funzionamento del sistema.

Dependability dei sistemi software – Sistemi socio-tecnici

- Società: leggi, regolamenti e cultura che influenzano il funzionamento del sistema.

- La seguente figura fornisce una rappresentazione grafica dei diversi livelli che costituiscono un sistema socio-tecnico:



- Ci sono interazioni e dipendenze tra i livelli di un sistema e i cambiamenti a un livello si ripercuotono sugli altri livelli.

- Nel contesto della dependability, una prospettiva di sistema è essenziale.

Dependability dei sistemi software – Proprietà emergenti

- Le proprietà emergenti sono proprietà del sistema nel suo insieme piuttosto che proprietà che possono essere derivate dalle proprietà dei componenti di un sistema.
- Le proprietà emergenti sono una conseguenza delle relazioni tra i componenti del sistema.
- Possono quindi essere valutate e misurate solo dopo che i componenti sono stati integrati in un sistema.
- Alcuni esempi di proprietà emergenti sono le seguenti:
  - Volume:
    - Il volume di un sistema (lo spazio totale occupato) varia a seconda di come i componenti sono disposti e collegati.
  - Reliability:
    - L'affidabilità (reliability) del sistema dipende dalla reliability dei componenti, ma interazioni impreviste possono causare nuovi tipi di guasti e quindi influenzare la reliability del sistema.

Dependability dei sistemi software – Proprietà emergenti

- Security:
  - La security del sistema (la sua capacità di resistere agli attacchi) è una proprietà complessa che non può essere facilmente misurata.
  - È possibile pensare ad attacchi non previsti dai progettisti del sistema; questi attacchi, quindi, possono superare le protezioni del sistema che erano state precedentemente previste.
- Repairability:
  - Questa proprietà riflette quanto sia facile riparare un problema con il sistema una volta che è stato scoperto.
  - Essa dipende dalla capacità di diagnosticare il problema, di accedere ai componenti difettosi e di modificare o sostituire questi ultimi.
- Usabilità:
  - Questa proprietà riflette quanto sia facile usare il sistema.
  - Essa dipende dai componenti tecnici del sistema, dai suoi operatori e dal suo ambiente operativo.

Dependability dei sistemi software – Proprietà emergenti

- Esistono due tipi di proprietà emergenti, ovvero:
  - Proprietà funzionali:
    - Queste si hanno quando tutte le parti di un sistema operano insieme per raggiungere qualche obiettivo.
    - Per esempio, una bicicletta ha la proprietà funzionale di essere un dispositivo di trasporto una volta che è stata assemblata dai suoi componenti.
  - Proprietà non funzionali:
    - Esempi sono la reliability, le performance, la safety e la security.
    - Queste riguardano il comportamento del sistema nel suo ambiente operativo.
    - Sono spesso critiche per i sistemi basati su computer, dato che il fallimento nel raggiungere qualche livello minimo definito in queste proprietà può rendere il sistema inutilizzabile.

Dependability dei sistemi software – Regolamentazione e compliance (conformità)

- Molti sistemi critici sono sistemi regolamentati, il che significa che il loro uso deve essere approvato da un regolatore esterno prima che i sistemi entrino in servizio.
  - Esempi: sistemi nucleari, sistemi di controllo del traffico aereo, dispositivi medici.
  - Un caso di safety e dependability deve essere approvato dal regolatore.
  - Quindi, chi sviluppa sistemi critici deve creare le prove per convincere un regolatore che il sistema è affidabile, sicuro e protetto.
- La regolamentazione e la compliance (seguire le regole) si applicano al sistema socio-tecnico nel suo insieme e non semplicemente ai singoli elementi software di quel sistema.
- I sistemi legati alla safety possono dover essere certificati come sicuri dal regolatore.
- Per ottenere la certificazione, le aziende che sviluppano sistemi critici per la safety devono produrre un ampio caso di safety che dimostri che le regole e i regolamenti sono stati seguiti.
- Sviluppare tutta la documentazione necessaria per la certificazione può essere tanto oneroso quanto lo è sviluppare il sistema stesso.

Dependability dei sistemi software – Ridondanza e diversità

- Il termine «Ridondanza» implica l'idea di mantenere più di una versione dei componenti critici in modo che se uno si guasta, è disponibile un backup.
- Il termine «Diversità» comporta l'idea di fornire la stessa funzionalità in modi diversi in componenti diversi in modo che non falliscano nello stesso modo.
- I componenti ridondanti e diversi dovrebbero essere indipendenti in modo che non subiscano guasti in «modo comune»
- Le attività di processo, come la validazione, non dovrebbero dipendere da un singolo approccio, come il test, per validare il sistema.
- Attività di processo ridondanti e diverse sono importanti specialmente per la verifica e la validazione.
- Attività di processo multiple e diverse che si completano a vicenda e permettono un controllo incrociato aiutano ad evitare errori di processo, che possono portare ad errori nel software.

Dependability dei sistemi software – Processi «dependable»

- Per assicurare un numero minimo di errori nel software, è importante avere un processo software ben definito e ripetibile.
- Un processo ripetibile ben definito è un processo che non dipende interamente dalle abilità individuali; piuttosto può essere attuato da diverse persone.
- I regolatori usano le informazioni sul processo per controllare se è stata usata una buona pratica di ingegneria del software.
- Per l'individuazione degli errori, è chiaro che le attività del processo dovrebbero includere uno sforzo significativo dedicato alla verifica e alla validazione.
- Due importanti caratteristiche di processo dependable (affidabile) sono le seguenti:
  - Deve essere esplicitamente definito:
    - Il processo deve avere un modello ben definito che sarà usato per guidare la produzione del software.
    - Durante il processo devono essere raccolti dati che provino che il team di sviluppo abbia seguito il processo come definito nel corrispettivo modello.

Dependability dei sistemi software – Processi «dependable»

- Deve essere ripetibile:
  - Il processo non si deve basare sull'interpretazione e sul giudizio individuale.
  - Il processo può essere ripetuto attraverso progetti e con diversi membri del team, indipendentemente da chi è coinvolto nello sviluppo.

- Le attività che è necessario eseguire per ottenere un processo dependable sono le seguenti:
  - Revisioni dei requisiti, per controllare che i requisiti siano, per quanto possibile, completi e coerenti.
  - Gestione dei requisiti, per assicurarsi che i cambiamenti sui requisiti siano controllati e che il loro impatto sia ben compreso.
  - Specificazione formale, in cui viene creato e analizzato un modello matematico del software.
  - Modellazione del sistema, tramite cui il progetto del software è esplicitamente documentato come un insieme di modelli opportuni, e i collegamenti tra i requisiti e questi modelli sono documentati.
  - Ispezioni del progetto e del programma, in cui le diverse descrizioni del sistema sono ispezionate e controllate da diverse persone.

Dependability dei sistemi software – Processi «dependable»

  - Analisi statica, in cui vengono effettuati controlli automatici sul codice sorgente del programma.
  - Pianificazione e gestione dei test, in cui si progetta un insieme completo di test di sistema.

- Il software dependable spesso richiede una certificazione, quindi deve essere prodotta una documentazione sia del processo che del prodotto.
- L'analisi anticipata dei requisiti è anche essenziale per scoprire requisiti e conflitti di requisiti che possono compromettere la sicurezza del sistema.
- Questi sono in conflitto con l'approccio generale nello sviluppo agile di co-sviluppo dei requisiti e del sistema e di minimizzazione della documentazione.
- Si può definire un processo agile che incorpora tecniche come lo sviluppo iterativo, lo sviluppo test-first e il coinvolgimento degli utenti nel team di sviluppo.
- Finché il team segue quel processo e documenta le sue azioni, i metodi agili possono essere usati.
- Tuttavia, la documentazione aggiuntiva e la pianificazione sono essenziali, quindi "l'agile puro" è impraticabile per l'ingegneria dei sistemi dependable.